

ACF – Komponenten-Lebenszyklus

Konsequenzen der neuen Smart-Pointer-Semantik
für Real- und Composite-Komponenten in icomp / ipackage

inkl. Composite-Faktorisierung: Alte vs. Neue Welt

(Ergänzung zum Hauptbericht)

Erstellt für:	Witold Gantke, Software-Architekt
Bezugsdokument:	ACF_Changes_Summary_Nov2025_May2026.pdf
Fokus:	icom, ipackage, istd::TInterfacePtr-Hierarchie, Composite-Factory
Datum:	10. Mai 2026

Inhaltsverzeichnis

1. Einleitung und Kernbotschaft
2. Eigentumsfluss durch den Lebenszyklus
3. Real-Komponenten – Lebenszyklus-Konsequenzen
4. Composite-Komponenten – Lebenszyklus-Konsequenzen
5. PopInterfacePtr() – Sicherer Übergang Real ↔ Composite
6. Composite-Faktorisierung: Alte Welt vs. Neue Welt ■ NEU
7. Konsequenzen für ipackage – Paketladen und Komponentenfabriken
8. Zusammenfassung der Lebenszyklus-Effekte

1. Einleitung und Kernbotschaft

Im Hauptbericht wurden die API-Änderungen an `TInterfacePtr`, `TUniqueInterfacePtr` und `TSharedPtrInterfacePtr` beschrieben. Dieses Dokument vertieft die **architektonischen Konsequenzen** dieser Änderungen für den Lebenszyklus von **Real-Komponenten** und **Composite-Komponenten** im Kontext von `icomp` und `ipackage`. Ein eigenes Kapitel (Kap. 6) widmet sich explizit der **Faktorisierung von Composite-Komponenten** mit direktem Vergleich alte vs. neue Welt.

Kernbotschaft: Die Pointer-Erweiterungen kodifizieren ein bislang implizites Eigentumsmodell explizit auf API-Ebene. Daraus ergeben sich vier wichtige Effekte: (a) der Übergang Factory → Composite ist nun explizit Move-basiert, (b) Composite-Komponenten können sicher als Interface extrahiert werden (`PopInterfacePtr()`, drei-Fall-Logik), (c) `FromUnique()` macht den Übergang von exklusivem zu geteiltem Besitz an genau einer Stelle erkennbar, und (d) `dynamicCast<T>()` erlaubt typsichere Downcasts ohne Ownership-Verlust – relevant für interface-orientierte Composite-Hierarchien.

2. Eigentumsfluss durch den Lebenszyklus

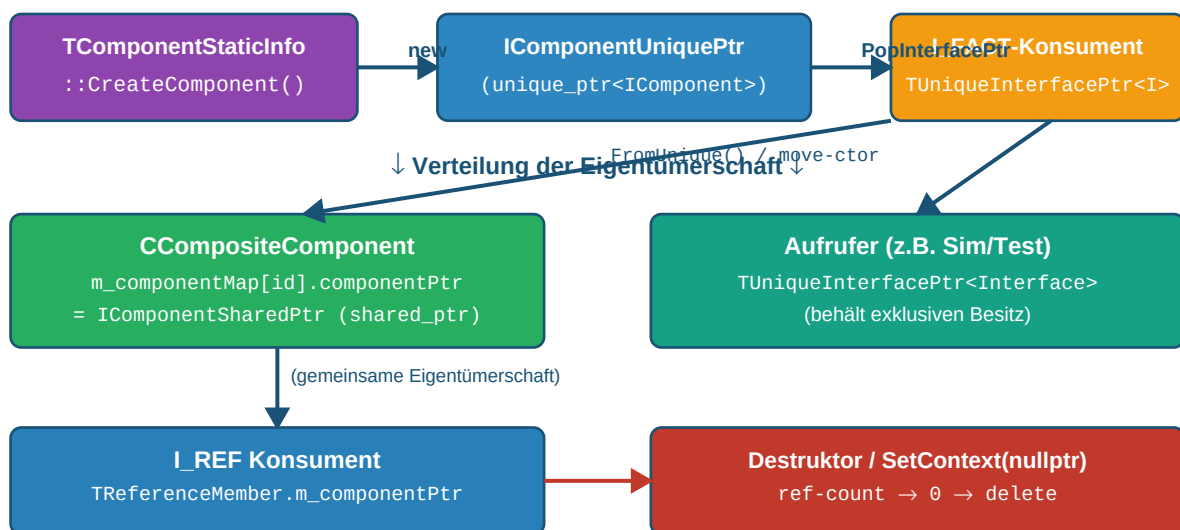
Der Eigentumsfluss einer Komponente folgt einem klar getrennten Drei-Stufen-Modell: **Erzeugung als Unique → Übergang zu Shared → laterale Sharing-Referenzen via I_REF**.

```
// Include/icomp/IComponent.h
typedef std::unique_ptr<IComponent> IComponentUniquePtr; // Erzeugung
typedef std::shared_ptr<IComponent> IComponentSharedPtr; // Speicherung

// Include/icomp/TFactoryMember.h (I_FACT)
istd::TUniqueInterfacePtr<Interface> CreateInstance(...) const;

// Include/icomp/TReferenceMember.h (I_REF)
mutable IComponentSharedPtr m_componentPtr; // shared mit Composite
```

Eigentumsfluss: Factory → Unique → Shared



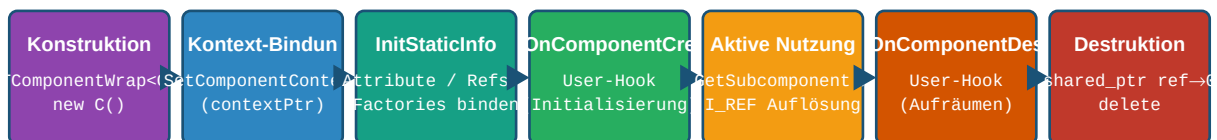
Konversion ist explizit (`FromUnique` / `std::move`) — keine impliziten Kopien.
`PopInterfacePtr()` unterstützt sowohl Real- als auch Composite-Komponenten (3 Fälle).

Phase	Vorher (implizit)	Nachher (explizit)	Konsequenz
Factory-Aufruf	Rückgabe-Eigentümerschaft unklar dokumentiert	TUniqueInterfacePtr<I> mit exklusiver Move-Semantik	Compiler erzwingt std::move – kein versehentliches Doppelt-Eigentum
Übergang zu Composite	Implicit copy-construction mit Refcount-Bump	FromUnique() oder Move-Ctor (rvalue erforderlich)	Eindeutige Übergabestelle, einfacher zu auditieren
I_REF-Auflösung	shared_ptr-Kopie	TSharedInterfacePtr mit dynamicCast<T>()	Typsichere Downcasts ohne Ownership-Verlust
Destruktion	Refcount → 0 → delete	Unverändert, aber durch Unique→Shared keine Doppelfreigabe	Die Move-Erzwingung schließt eine Klasse von Lebenszyklus-Bugs

3. Real-Komponenten – Lebenszyklus-Konsequenzen

Real-Komponenten sind Blattknoten der Komponentenhierarchie – sie kapseln die eigentliche Funktionalität und besitzen keine Sub-Komponenten. Klassen wie `CComponentBase` und ihre direkten Ableitungen (z.B. Logger-Komponenten, Datei-I/O-Komponenten) gehören dazu.

Lebenszyklus-Phasen einer Komponente



Pointer-Typ pro Phase:

1: Konstruktion	raw new (im Wrap)
2-3: Kontext + Init	TUniqueInterfacePtr — exklusiv im Factory-Aufrufer
4-5: Eigentumsübergang	FromUnique() / std::move → TSharedInterfacePtr
6-7: Sharing & Destruktion	Mehrere shared_ptr-Halter (Composite + I_REF) → ref→0

3.1 Erzeugung über die Factory

Die Erzeugung erfolgt zentral in `TComponentStaticInfo`:

```
IComponentUniquePtr TComponentStaticInfo<Component>::CreateComponent() const {  
    auto* componentPtr = new TComponentWrap<Component>();  
    return IComponentUniquePtr(componentPtr);  
}
```

Die Real-Komponente liegt nach Konstruktion **noch nicht aktiviert** vor: weder ein Kontext-Pointer ist gesetzt, noch wurde `OnComponentCreated()` aufgerufen. Beides geschieht erst in `TComponentWrap::SetComponentContext()` *nachdem* der `TUniqueInterfacePtr`-Eigentümer die Komponente an einen Halter (Composite oder Test-Code) übergeben hat.

Architektonische Konsequenz: Mit der erzwungenen Move-Semantik ist die Reihenfolge (Konstruktion → Kontext → Hook) jetzt durch das Typsystem geschützt. Kopien sind verboten, weil `TUniqueInterfacePtr` nicht kopierbar ist; ein irrtümliches doppeltes `OnComponentCreated()` kann nicht mehr durch versehentliche Doppelregistrierung in zwei Composites entstehen.

3.2 Übergang Unique → Shared (Eintritt in den Composite)

Der entscheidende Eigentumsübergang findet in `CCompositeComponent.cpp` statt:

```
IComponentUniquePtr realComponentPtr =  
    realComponentInfoPtr->CreateComponent();  
componentInfo.componentPtr = std::move(realComponentPtr); // Unique → Shared
```

3.3 Destruktion einer Real-Komponente

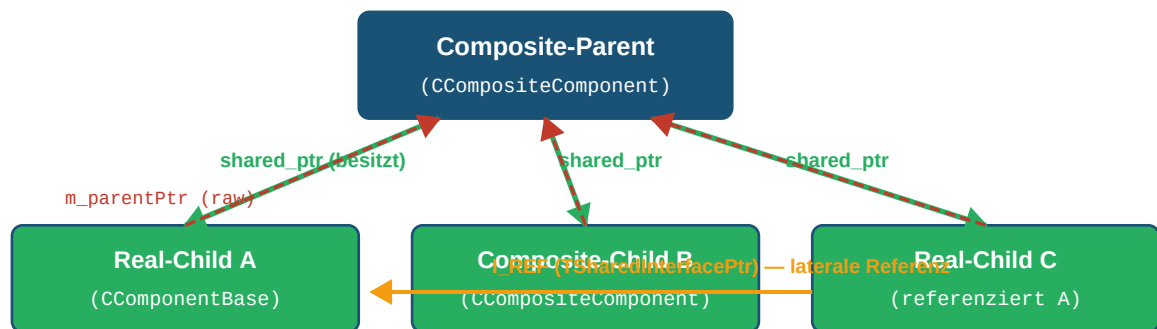
Eine Real-Komponente wird zerstört, sobald der letzte `shared_ptr` auf sie freigegeben wird. Üblicherweise sind das (a) der Eintrag im `CCompositeComponent::m_componentMap` des Parents und (b) ggf. `I_REF`-Caches anderer Komponenten.

Risiko: Wenn eine `I_REF`-haltende Komponente überlebt, nachdem der Parent-Composite zerstört wurde, bleibt die referenzierte Real-Komponente künstlich am Leben. Mit der neuen `dynamicCast<T>()`-API kann ein `I_REF`-Halter beim Zugriff prüfen, ob die referenzierte Komponente noch ihre erwartete Schnittstelle bedient.

4. Composite-Komponenten – Lebenszyklus-Konsequenzen

Composite-Komponenten (CCompositeComponent) verwalten eine Map untergeordneter Komponenten und delegieren Sub-Adressen. Sie sind die **einzigen Eigentümer** ihrer direkten Kinder.

Eigentums-Topologie und Zyklenvermeidung



Legende:

- shared_ptr — Eigentümerschaft (Parent → Child)
 - - -> Raw-Pointer — KEINE Eigentümerschaft (Child → Parent) → bricht Zyklen
 - I_REF (TSharedInterfacePtr) — laterale Geschwister-Referenz, gemeinsame Eigentümerschaft
- Konsequenz: Keine Zyklen — Lebenszeit deterministisch durch Parent-Hierarchie.

4.1 Eigentums-Topologie

Beziehung	Pointer-Typ	Eigentümerschaft	Zyklen-Risiko
Composite → Real-Child	shared_ptr<IComponent>	Composite besitzt	Keines (Top-Down)
Composite → Composite-Child	shared_ptr<IComponent>	Parent besitzt	Keines (Top-Down)
Child → Parent	raw IComponent*	KEINE Eigentümerschaft	Keines (raw bricht Zyklus)
I_REF (Geschwister)	TSharedInterfacePtr<I>	Geteilt mit Composite	Möglich, falls Geschwister sich gegenseitig referenzieren
I_FACT-Resultat	TUniqueInterfacePtr<I>	Aufrufer besitzt	Keines

Architektur-Designentscheidung: Es wird bewusst *kein* `std::weak_ptr` in der Komponentenhierarchie verwendet. Stattdessen löst die Asymmetrie *Parent: shared / Child: raw* das klassische Zyklen-Problem.

4.2 Sub-Komponenten-Zugriff: Get vs. Create

Methode	Rückgabe-Pointer	Eigentümerschaft	Anwendungsfall
GetSubcomponent(id)	IComponentSharedPtr	Geteilt – gleiches Objekt	Lookup einer registrierten Sub-Komponente; identische Identität bei wiederholtem Zugriff
CreateSubcomponent(id)	IComponentUniquePtr	Aufrufer-exklusiv	Neue Instanz nach Vorlage; Composite cached sie nicht

4.3 Destruktion eines Composite

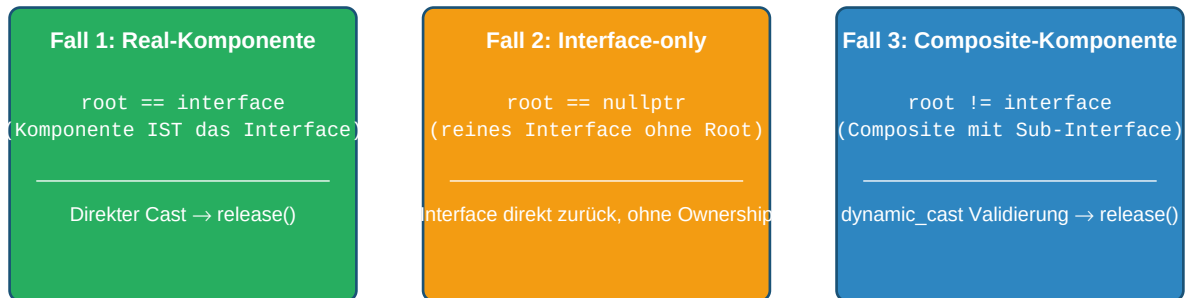
```
for (auto& info : m_componentMap) {
    info.componentPtr->SetComponentContext(nullptr, nullptr, false);
    info.componentPtr.reset();
}
```

Existieren noch externe I_REF-Halter auf eine Sub-Komponente, übersteht diese den Reset zwar, ist aber **kontextlos**. Mit `dynamicCast<T>()` kann ein Halter einen sanften `nullptr`-Pfad einbauen, statt einen Crash zu produzieren.

5. PopInterfacePtr() – Sicherer Übergang Real ↔ Composite

Eine der substantiellsten Erweiterungen ist die intelligente PopInterfacePtr()-Methode in TUniqueInterfacePtr. Sie überträgt das Eigentum als rohen Pointer und unterscheidet drei Fälle:

PopInterfacePtr() – drei Fälle (Real vs. Composite)



Vor der Erweiterung: nur Fall 1 sicher unterstützt.

Mit den Erweiterungen: Composite-Fall via dynamic_cast validiert →
kein UB mehr bei Interface-Extraktion aus Composites.

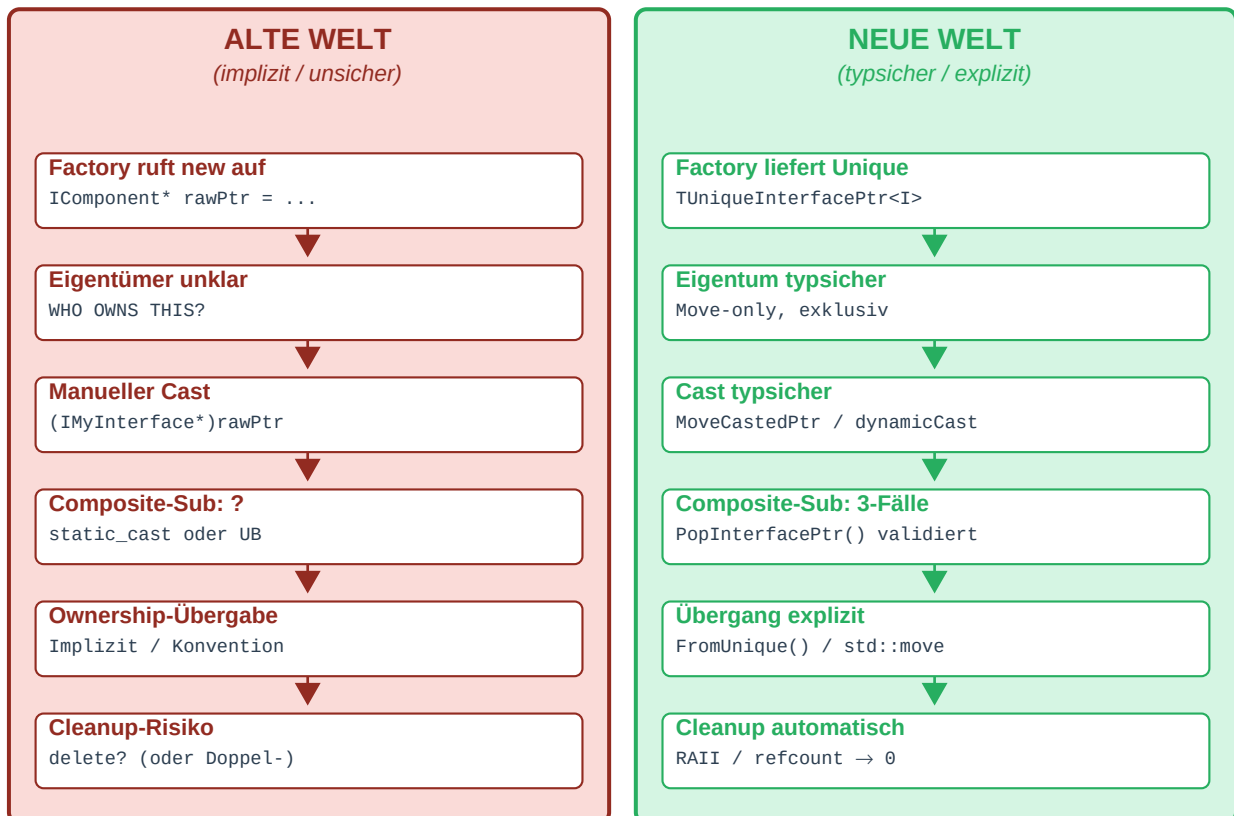
Eine Composite-Komponente exportiert in der Regel *nicht das Composite selbst*, sondern eine bestimmte Schnittstelle. PopInterfacePtr() nutzt im Composite-Fall einen dynamic_cast-validierten Schritt, um sicherzustellen, dass das gepoppte Interface tatsächlich vom Root-Objekt bedient wird.

```
// Include/icomp/TSimComponentsFactory.h:121
return icomp::IComponentUniquePtr(
    CreateInstance(componentId).PopInterfacePtr());
```

6. Composite-Faktorisierung: Alte Welt vs. Neue Welt

Dieses Kapitel beleuchtet explizit, wie die Faktorisierung – also der Akt, über eine Factory-API eine Composite-Komponente zu erzeugen, vorzukonfigurieren und an einen Aufrufer zu übergeben – sich vor und nach den Smart-Pointer-Erweiterungen darstellt. Composite-Faktorisierung ist deutlich anspruchsvoller als Real-Faktorisierung, weil **Root-Objekt** und **exportiertes Interface auseinanderfallen**: das Composite selbst (CCompositeComponent) implementiert nicht das Anwender-Interface – dieses wird durch eine Sub-Komponente bzw. durch die Registry-Mappings bedient.

Composite-Faktorisierung: Alte vs. Neue Welt



Zentrale Erkenntnis: Der Compiler erzwingt jetzt das, was vorher nur durch Disziplin lief.

6.1 Alte Welt – die Schmerzpunkte

In der alten Welt (vor PR #184 "TInterfacePtr enhancements" und PR #25 "prefer rvalue semantic") war die Composite-Faktorisierung ein Bereich mit mehreren impliziten Annahmen, die nur durch Konvention (und Code-Reviews) abgesichert waren. Typische Pre-Erweiterungs-Muster sahen so aus:

```
// ALTE WELT – Skizze des typischen Aufrufmusters
IComponent* rawPtr = factory->CreateComponent(); // IComponentUniquePtr
// ... welche Schnittstelle? Wer wandelt um?
IExportedInterface* iface =
    dynamic_cast<IExportedInterface*>(rawPtr); // manuell
// ...arbeiten...
// Wer hält wen am Leben? Composite? Aufrufer?
// std::auto_ptr / unique_ptr / raw – uneinheitliche Speicherung
```

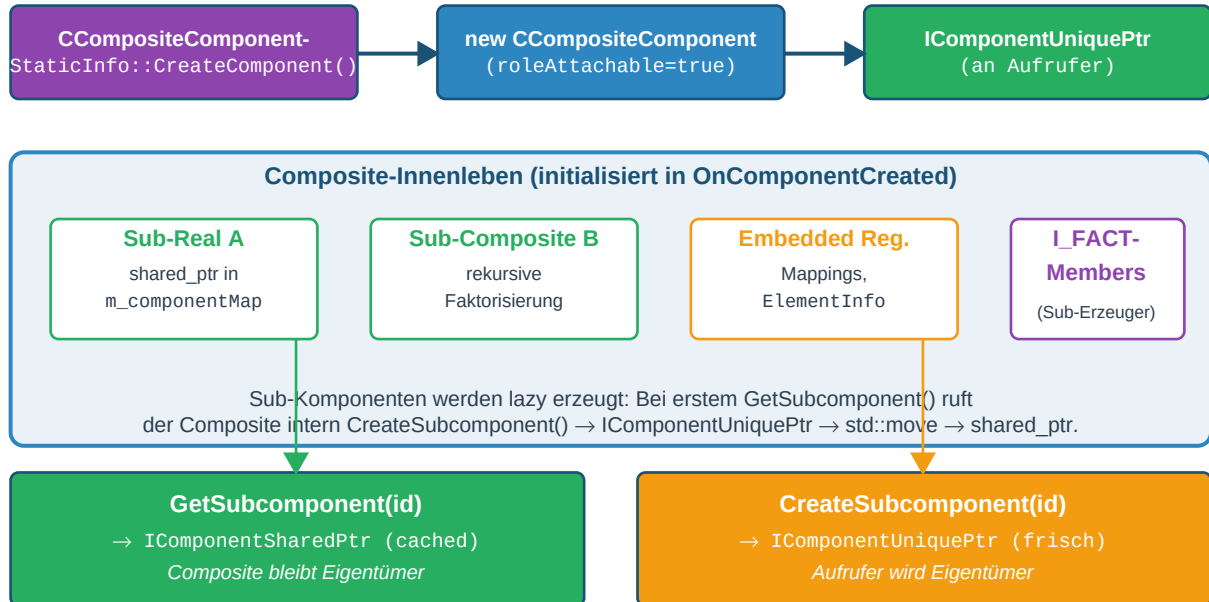
#	Problem in der alten Welt	Auswirkung auf Composite
1	Implicit copy-construction Unique→Shared	Versehentliche zweite shared_ptr-Kette → doppelte Destruktion bei Composite-Reset
2	static_cast für Composite-Sub-Interfaces	Undefined Behavior, wenn Mapping in der Registry nicht so gelesen wurde, wie der Aufrufer annahm
3	Keine standardisierte Fabrik-Rückgabe	Manche Stellen lieferten Root-Objekt, andere Sub-Interface – Aufrufer mussten unterscheiden
4	Keine Move-Erzwingung	Eine im Composite gespeicherte Sub-Komponente konnte versehentlich kopiert (statt gemoved) werden → zwei Composites teilten sich denselben Sub-Knoten
5	I_REF auf Composite-Sub: ungeprüfter Cast	Nach Paket-Reload (z.B. Compositor) konnte das Sub-Interface entfallen sein – Crash statt nullptr
6	PopInterfacePtr() ohne Composite-Validierung	Beim Übergang aus generischen Containern (TSimComponentsFactory) konnte das exportierte Interface nicht zuverlässig vom Composite-Root extrahiert werden

Konkretes Beispiel: Im Compositor (AcfSIn) führte das Hot-Reload eines Pakets dazu, dass I_REF-Halter auf Composite-Sub-Interfaces über einen unsicheren static_cast zugegriffen, was nach Paket-Wechsel zu Speicher-Verletzungen führte. Diese Klasse von Bugs ließ sich nur durch ein robusteres Pointer-Modell systematisch ausschließen.

6.2 Neue Welt – die kanonische Composite-Faktorisierung

Mit den erweiterten Smart-Pointer-APIs gibt es jetzt einen klaren, vom Typsystem erzwungenen Pfad. Drei Ebenen sind zu unterscheiden:

Composite-Faktorisierung – innere Mechanik (NEU)



(a) Erzeugung des Composite-Roots

```
// Include/icmp/CCompositeComponentStaticInfo.cpp:175-178
IComponentUniquePtr CCompositeComponentStaticInfo::CreateComponent() const {
    return IComponentUniquePtr(new CCompositeComponent(true));
}
```

Das Composite wird als `IComponentUniquePtr` zurückgegeben. Sub-Komponenten existieren zu diesem Zeitpunkt *noch nicht* – sie werden lazy beim ersten `GetSubcomponent()`-Aufruf instanziiert.

(b) Faktorisierung über I_FACT auf einem Composite

```
// Include/icmp/TFactoryMember.h:101-136
istd::TUniqueInterfacePtr<Interface>
TFactoryMember<Interface>::CreateInstance(...) const {
    IComponentUniquePtr newComponentPtr = CreateComponent();
    Interface* retVal = ExtractInterface<Interface>(newComponentPtr.get());
    return istd::TUniqueInterfacePtr<Interface>(
        newComponentPtr.release(), retVal);
}
```

Der Rückgabewert ist ein `TUniqueInterfacePtr<Interface>`, der intern *Root-Objekt und Interface-Pointer separat* hält. Damit weiß der Aufrufer: "ich besitze die ganze Composite-Wurzel, sehe sie aber durch das Interface-Fenster". Die Composite-Sub-Komponenten werden durch den Composite selbst lazy instanziiert; die Faktorisierung des Roots ist eine atomare Operation.

(c) Faktorisierung über TSimComponentsFactory

```
// Include/icmp/TSimComponentsFactory.h:113-122
icmp::IComponentUniquePtr TSimComponentsFactory<Base>::
    CreateSubcomponent(const QByteArray& id) const {
```

```

auto componentPtr = m_base.CreateSubcomponent(id);
if (componentPtr != nullptr) return componentPtr;
return icip::IComponentUniquePtr(
    CreateInstance(id).PopInterfacePtr()); // 3-Fall-Logik
}

```

Hier kommt die **drei-Fall-Logik von** `PopInterfacePtr()` voll zum Tragen: Wenn `CreateInstance()` ein `TUniqueInterfacePtr<Interface>` auf einem *Composite-Root* liefert, validiert `PopInterfacePtr()` via `dynamic_cast`, dass das Composite tatsächlich das Interface bedient, bevor es das Eigentum als rohen Pointer in das umhüllende `IComponentUniquePtr` übergibt. Der Bugfix in Commit `ece6d3d2` ("TSimComponentsFactory fixed") betraf genau diesen Pfad.

6.3 Direkter Vergleich der Aufrufseite

Aspekt	Alte Welt	Neue Welt
Factory-Rückgabe	Raw-Pointer oder QSharedPointer ohne Interface-Trennung	<code>TUniqueInterfacePtr<I></code> mit Root/Interface-Trennung
Composite-Sub-Interface-Cast	<code>static_cast</code> / manuelles <code>dynamic_cast</code>	<code>PopInterfacePtr()</code> validiert via <code>dynamic_cast</code> intern
Eigentumsübergang an Composite	Implicit <code>shared_ptr</code> -Konstruktion (RefCount-Bump)	<code>FromUnique()</code> / <code>std::move</code> — explizit, rvalue-erforderlich
Konvertierung Composite-Root → Sub-Interface	manuell, fehleranfällig	<code>MoveCastedPtr()</code> / <code>dynamicCast<T>()</code> — typvalidiert
Wiederverwendung des Aufrufpfads	Pro Klasse Custom-Code	Einheitlich für Real und Composite
Test-Code (TSimComponentsFactory)	Spezialfall-Logik notwendig	Generischer Pfad über <code>PopInterfacePtr()</code>
Hot-Reload-Sicherheit (Compositor)	Crash bei verlorenem Sub-Interface	<code>dynamicCast<T>()</code> liefert <code>nullptr</code> → Halter behandelt sanft

Architektonische Bilanz: Composite-Faktorisierung ist in der neuen Welt eine **generische, typvalidierte Operation**. Was vorher pro Aufrufstelle individuelle Sorgfalt erforderte (Cast, Ownership, Lifetime), ist jetzt in den Smart-Pointer-Operationen konsolidiert. Der gleiche Code-Pfad funktioniert für Real- und Composite-Komponenten, weil die Pointer-Klassen den Unterschied zwischen "Root == Interface" und "Root != Interface" intern transparent handhaben. Das ist eine substantielle Reduktion der konzeptionellen Last für Komponenten-Autoren.

7. Konsequenzen für ipackage – Paketladen und Komponentenfabriken

Pakete (IPackage) sind die Eintrittspunkte, über die Komponentenklassen in den ACF-Laufzeitkontext eingespeist werden. Pro exportierter Klasse stellt ein Paket eine TComponentStaticInfo-Instanz bereit, deren CreateComponent() die Real- oder Composite-Instanz als TUniqueInterfacePtr liefert.

Aspekt	Pre-Erweiterung	Post-Erweiterung	Architekturwirkung
Komponenten-Erzeugung in IPackage	unique_ptr-Rückgabe	TUniqueInterfacePtr mit Move-Semantik	Paket-API ist robuster gegen unbeabsichtigtes Kopieren
Paket-Entladen (Reload)	Refcount muss 0 erreichen, sonst DLL-Lock	Unverändert, aber dynamicCast erlaubt sanftere Halter-Migration	Heißes Reladen von Paketen leichter zu auditieren
Cross-Package-Referenzen	TSharedInterfacePtr (impliziter Cast)	TSharedInterfacePtr::dynamicCast<T>()	Typ-Mismatches zwischen Paket-Versionen werden erkannt
Sim/Test-Konsumenten	Manuell konstruiertes shared_ptr	FromUnique() / CreateFromUnique()	Eindeutige API-Stelle für Lebenszeit-Übergänge
Composite-Export aus Paket	Aufrufer musste Sub-Interface manuell erkennen	PopInterfacePtr() liefert validiertes Interface	Paketgrenzen-überschreitende Composite-Nutzung typsicher

Hot-Reload-Szenario (Compositor): Der Compositor lädt und entlädt Komponenten-Pakete zur Laufzeit. Mit dem neuen dynamicCast<T>() können I_REF-Halter, die einen alten Paket-Stand referenzieren, beim erneuten Zugriff feststellen, dass die erwartete Schnittstelle nach Reload nicht mehr verfügbar ist – ohne Crash, mit definiertem nullptr-Pfad. Das war ein realer Schmerzpunkt vor den Erweiterungen.

8. Zusammenfassung der Lebenszyklus-Effekte

Pointer-Erweiterung	Wirkung auf Real-Komponenten	Wirkung auf Composite-Komponenten
Move-Semantik (rvalue)	Erzwingt eindeutigen Eigentumsübergang Factory→Composite; verhindert Doppel-Destruktion.	Sub-Komponenten-Übergabe an m_componentMap ist als Move erkennbar; Code-Review erleichtert.
FromUnique() / CreateFromUnique()	Benannter Übergangspunkt Unique→Shared; vereinfacht Audit von Lebenszyklen.	Composite-Roots können explizit als Shared in Container eingespeist werden.
dynamicCast<T>()	I_REF-Halter prüfen Schnittstellen-Verfügbarkeit zur Laufzeit (z.B. nach Paket-Reload).	Composite-Roots können auf Sub-Interfaces gecastet werden ohne Ownership zu verlieren.

Pointer-Erweiterung	Wirkung auf Real-Komponenten	Wirkung auf Composite-Komponenten
PopInterfacePtr() drei-Fall-Logik	Fall 1 (root==interface) deckt einfache Real-Komponenten ab.	Fall 3 mit dynamic_cast deckt Composite→Sub-Interface sicher ab; eliminiert UB.
Derived-Type-Konstruktor en	Implizite Aufwärtskonvertierung für I_REF<DerivedInterface>; weniger Boilerplate.	Composite kann Sub-Komponenten als Basistyp speichern und derived behandeln.
nullptr-Vergleich + get()	Sauberer Defensivcode in OnComponentCreated/Destroyed.	Vereinfachte Iteration über m_componentMap-Einträge.
MoveCastedPtr()	Selten relevant für Real-Komponenten.	Composite-Faktorisierung mit type-validiertem Move; Aufrufer gewinnt typgenaues Eigentum.

Architektonisches Fazit: Die Pointer-Änderungen erweitern das ACF-Komponentenmodell nicht um neue Lebenszyklus-Phasen, sondern **kodifizieren das bestehende Modell explizit** auf API-Ebene. Das Modell *Factory* ⇒ *Unique* ⇒ *Composite-shared* ⇒ *I_REF-shared* ⇒ *Refcount-Destruktion* war zuvor durch Konventionen abgesichert; jetzt erzwingt das Typsystem dieselben Garantien zur Compile-Zeit. Speziell für die **Composite-Faktorisierung** bedeutet das: ein einheitlicher Aufrufpfad, der Real- und Composite-Komponenten mit derselben Sorgfalt behandelt, ohne dass Aufrufer den Unterschied kennen müssen. Für einen Software-Architekten heißt das: **weniger Reviewer-Mental-Load, robusteres Hot-Reload-Verhalten, und ein sauberes Fundament für tiefe Composite-Hierarchien.**

Dieses Dokument ergänzt den Hauptbericht ACF_Changes_Summary_Nov2025_May2026.pdf um eine Vertiefung der Lebenszyklus-Aspekte und der Composite-Faktorisierung. Quellen: Include/istd/TInterfacePtr.h, Include/icomp/{IComponent, CCompositeComponent, CCompositeComponentStaticInfo, TComponentStaticInfo, TFactoryMember, TReferenceMember, TSimComponentsFactory}.h/.cpp, Include/ipackage/IPackage.h. Bezugs-PRs: ACF #184, AcfSIn #25.